Gaussian Processes

Bayesian Modeling with Distributions over Functions

Max Kieff, Mengyue Huang, Alan Wu, Yaohao Zhang, Andrew Zhao

The Shortcomings of Regression

How can we do Bayesian inference when we are unsure of the functional form?

Ordinary regression assumes the response is linear in the features.

BAYESIAN LINEAR REGRESSION

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i,\ \varepsilon_i \sim N(0, \sigma^2),\ \beta \sim p(\beta)$$

WHERE THIS FALLS SHORT

Functional form chosen in advance

Must commit to linear, quadratic, cubic, etc before seeing the data.

Misspecification risk

If the truth isn't in the chosen class, fit and inference degrade.

Limited flexibility

A finite coefficient vector may not capture complex nonlinear structure.

Uncertainty on the wrong object

We get uncertainty on β, but our real unknown is the function itself.

Why Gaussian Processes?

What if we want a prior over functions, not parameters?

Parametric approach

Assume a fixed form, e.g.

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \varepsilon$$

- Choose degree / basis upfront
- Model misspecification if wrong
- Hard to express “smooth but unknown”

Gaussian Process approach

Place a prior directly on *f*:

$$y = f(x) + \varepsilon \quad where \quad f \sim GP(m, k)$$

- Nonparametric: no fixed basis
- Encodes smoothness via kernel
- Gives structured uncertainty while remaining very flexible

What Is a Gaussian Process?

DEFINITION

A stochastic process $\{f(x) : x \in \mathcal{X}\}$ is a **Gaussian process** if, for every finite collection of inputs $x_1, \ldots, x_n \in \mathcal{X}$, the vector $(f(x_1), \ldots, f(x_n))$ has a multivariate normal distribution. The multivariate normal is governed by a **mean function** $m(x)$ and a covariance matrix that comes from the **kernel function** $k(x, x')$.

$$f \sim \mathcal{GP}\,(\,m(\cdot),\ k(\cdot, \cdot)\,)$$

At any finite set of inputs

$$(f(x_1), \ldots, f(x_n)) \sim \mathcal{N}(m, K)$$

with $(m)_i = m(x_i)$ and $(K)_{ij} = k(x_i, x_j)$.

Two ingredients

- $m(x) = \mathbb{E}[f(x)]$ — prior average behavior
- $k(x, x') = \mathrm{Cov}(f(x), f(x'))$ encodes smoothness & scale

The conceptual shift: from a prior on coefficients to a prior on functions.

The RBF (Squared Exponential) Kernel

The kernel determines how strongly two function values are related under the prior.

$$k(x, x') = \alpha^2 \exp\left(-\|x - x'\|^2 / 2\ell^2\right)$$

α² — amplitude

$k(x, x) = \alpha^2$ — marginal variance

Typical vertical size of function fluctuations.

ℓ — lengthscale

ℓ — rate of correlation decay

How far the influence of one point reaches.

IF x AND x′ ARE CLOSE

$k(x, x') \approx \alpha^2$ — strong correlation

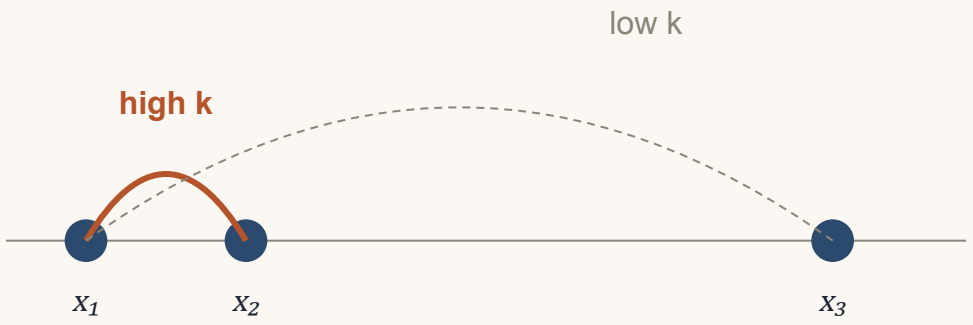
Nearby inputs → similar function values.

IF x AND x′ ARE FAR APART

$k(x, x') \approx 0$ — nearly independent

Distant inputs provide little info about each other.

HOW IT SHAPES THE PRIOR



Bayesian GP Regression

PRIOR ON THE FUNCTION

$$f \sim \mathcal{GP}(m, k_\theta)$$

θ = (α, ℓ) are the kernel hyperparameters.

LIKELIHOOD

$$y_i \mid f(x_i), \sigma^2 \sim \mathcal{N}(f(x_i), \sigma^2)$$

Gaussian noise: y = f(x) + ε, with ε ~ $\mathcal{N}$(0, σ²).

HYPERPRIORS

$$\alpha \sim p(\alpha), \ell \sim p(\ell), \sigma^2 \sim p(\sigma^2)$$

Fully Bayesian: priors on the kernel params and noise.

Hierarchical: $\alpha, \ell, \sigma^2 \to f \to y$

POSTERIOR PREDICTIVE AT A NEW INPUT · $f(x^*) \mid y \sim \mathcal{N}(\mu^*, v^*)$

POSTERIOR MEAN

$$\mu^* = m(x^*) + k^{*T}(K_\theta + \sigma^2 I)^{-1}(y - m)$$

*estimated function value at x**

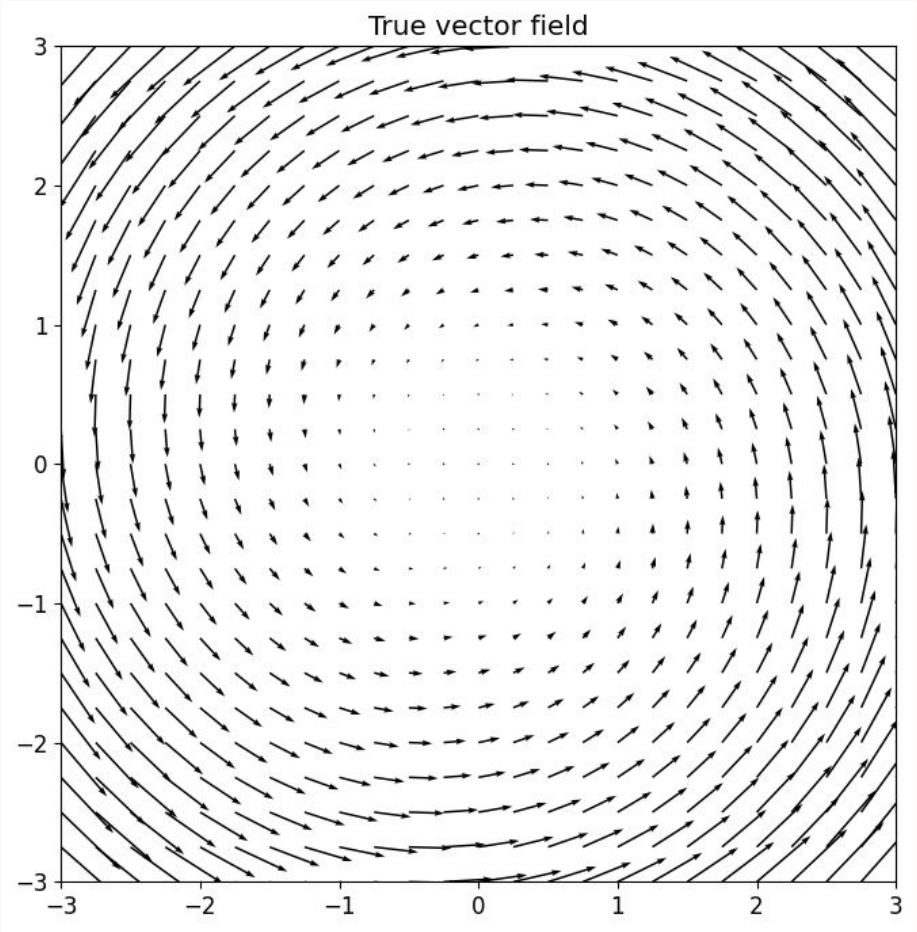
POSTERIOR VARIANCE

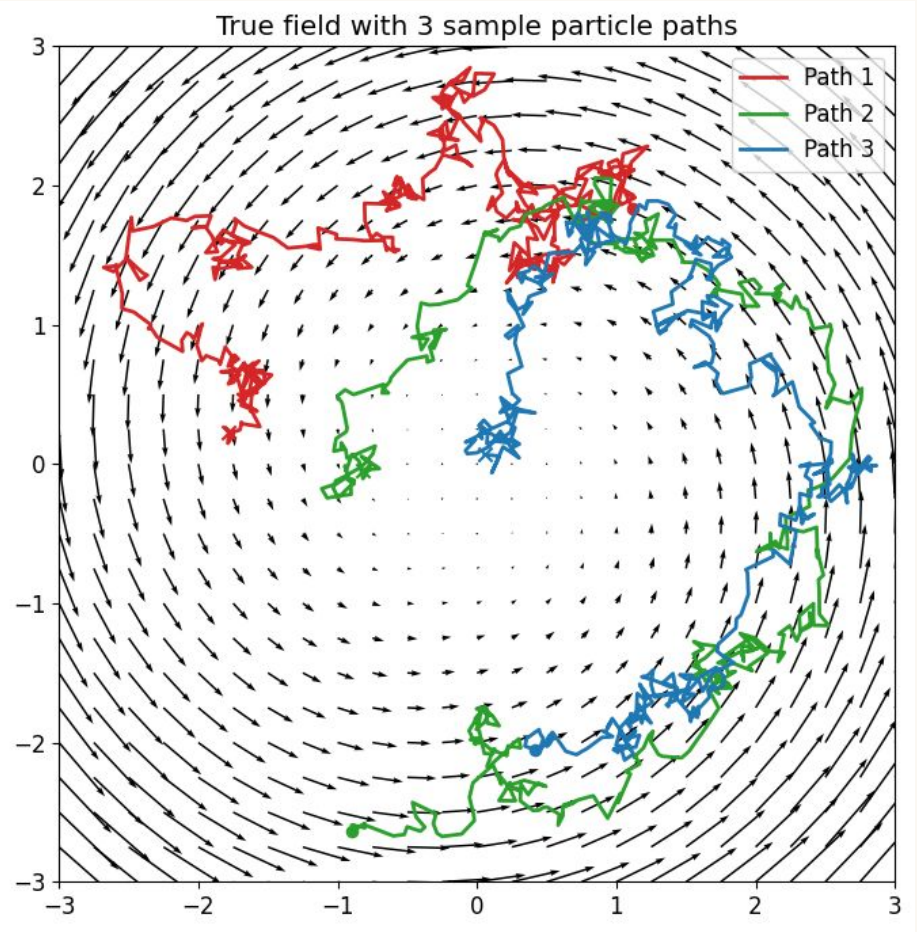
$$v^* = k(x^*, x^*) - k^{*T}(K_\theta + \sigma^2 I)^{-1}k^*$$

principled uncertainty — shrinks near data

Toy Example: Recovering a Hidden Drift Field

Particles in $\mathbb{R}^2$ do a random walk (Gaussian noise) and are driven by unknown vector field F. We only observe the particle trajectories; we want to recover F.





Toy Example: Recovering a Hidden Drift Field

Particles in $\mathbb{R}^2$ do a random walk (Gaussian noise) and are driven by unknown vector field F. We only observe the particle trajectories; we want to recover F.

$$X_{\square+1} = X_{\square} + F(X_{\square})\,\Delta t + \sigma\sqrt{\Delta t}\,\varepsilon_{\square} \quad \varepsilon_{\square} \sim N(0, I_2)$$

- $X_{\square}$: current particle position
- $\sigma\sqrt{\Delta t}\,\varepsilon_{\square}$: random perturbation over time Δt
- $F(X_{\square})\Delta t$: deterministic drift from the hidden field over time Δt
- $\sigma > 0$: sets the noise magnitude

STEP 1 — Finite-difference velocity

$$Y_{\square} := (X_{\square+1} - X_{\square}) / \Delta t$$

Rearranging the update yields $Y_{\square} = F(X_{\square}) + (\sigma/\sqrt{\Delta t})\,\varepsilon_{\square}$.

STEP 2 — A regression problem emerges

$$Y_i = F(x_i) + \eta_i$$

Each displacement is a **noisy local clue** about F. A GP prior on F stitches them together.

Fitting the GP to the Toy Example

The unknown object is the hidden vector field F; σ is treated as known.

THE DATA · Pairs (x_i, Y_i) with $x_i, Y_i \in \mathbb{R}^2$ — a position and its noisy local velocity $Y_i = (X_{i+1} - X_i) / \Delta t$.

Each short displacement is a noisy clue about F at the current location.

LIKELIHOOD

$$Y_i \mid F(x_i), \sigma^2 \sim \mathcal{N}\left(F(x_i), (\sigma^2/\Delta t)\, I_2\right)$$

Velocity centered on the true field; noise inflated by 1/Δt.

GP PRIOR ON THE FIELD

$$F_1 \sim \mathcal{GP}(0, k_\theta), \;\; F_2 \sim \mathcal{GP}(0, k_\theta)$$

Two independent components, zero mean, shared RBF kernel $\theta = (\alpha, \ell)$.

- *Base assumption is m(x)=0 everywhere (purely random walk)*

HYPERPRIORS

$$\alpha \sim \text{Half-Normal}(2.5), \ell \sim \text{Half-Normal}(2)$$

Weakly informative:

- *ℓ : for a 6x6 observation window, allows for some local variation*
- *α : forces are same order of magnitude as particle stdv (0.5)*

POSTERIOR TARGET

$$p(F_1, F_2, \alpha, \ell \mid Y_1, \ldots, Y_n)$$

Update the prior over smooth vector fields using the observed local velocities.

IMPLEMENTATION · Full Bayesian fit with **PyMC** (GP priors + MCMC)

How the Posterior Sharpens with More Data

10 trajectories

- Posterior is still diffuse.
- Main directional signal detected, estimate of “center” is off.

20 trajectories

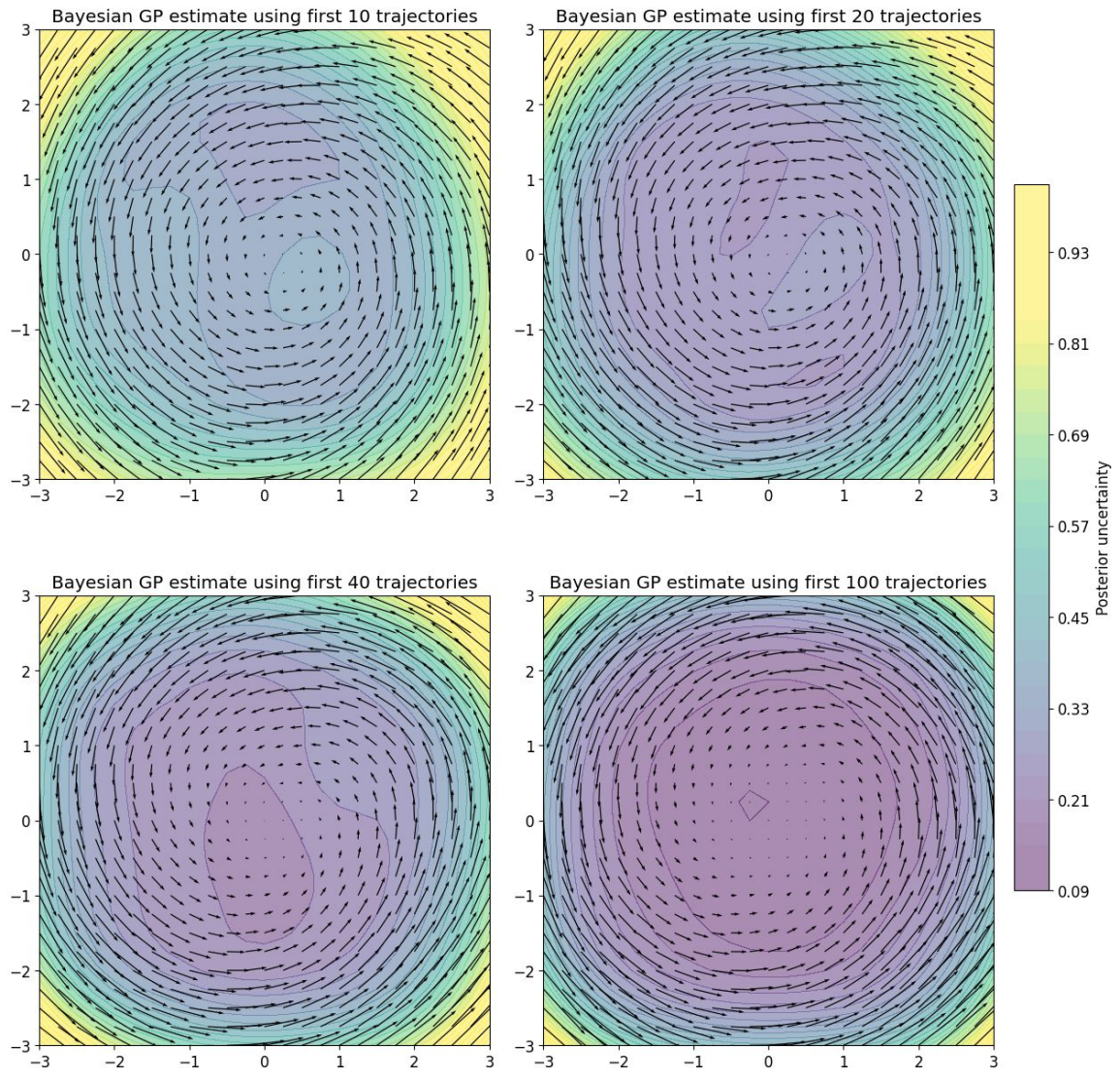
- Data are now strong enough to distinguish signal from noise in most of the interior.
- Posterior mean field is becoming stable.

40 trajectories

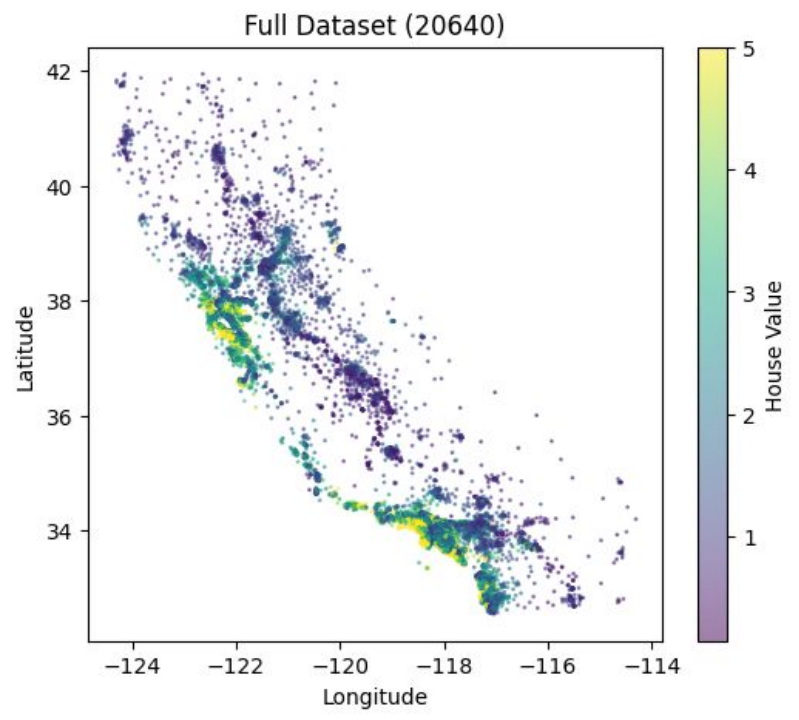
- Significant uncertainty remains around edges, reflecting lack of data.
- Global structure has converged.

100 trajectories

- Posterior has concentrated strongly around the true field.
- Posterior uncertainty reduced by an order of magnitude on majority of domain.



Case Study: California Housing Dataset



Full Dataset (N=20640)

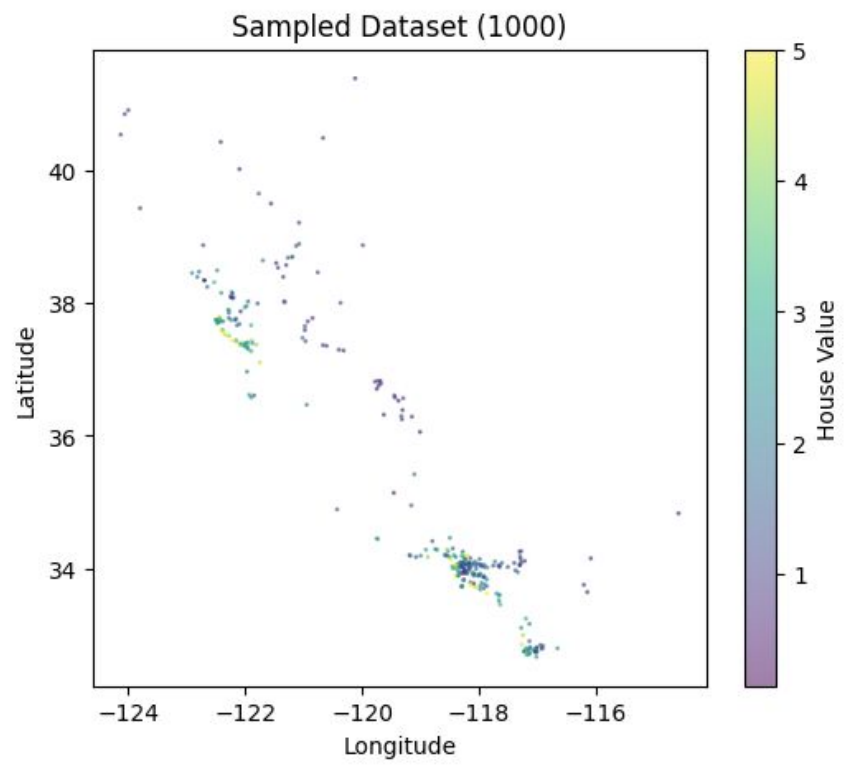
Dataset Overview

Based on the 1990 U.S. Census. Each row is a "block group" (600–3,000 people); target is median house value.

Column	Type	Description
`MedInc`	Float	Median income in the block group
`HouseAge`	Float	Median house age in the block group
`AveRooms`	Float	Average number of rooms per household
`AveBedrms`	Float	Average number of bedrooms per household
`Population`	Float	Total block group population
`AveOccup`	Float	Average number of household members
`Latitude`	Float	**Spatial input (X_2): block group latitude**
`Longitude`	Float	**Spatial input (X_1): block group longitude**
`MedHouseVal`	Float	**Target: median house value (×$100,000)**

Spatial inputs highlighted in blue; target variable highlighted in tan.

Normalization & Feature Selection



Training Dataset (N=1000)

Normalization: We normalized the train and test datasets by taking the log(Price) and z-scoring.

Feature Selection: We have eight features available but used only longitude and latitude.

- **Inductive Bias:** What scale should relate longitude, latitude, house age and median income? Using the RBF kernel for all of these implies they are comparable to one another in a meaningful way.
- **Curse of Dimensionality:** 8-dimensional data is sparse and would require a more flexible kernel.
- **Computation:** Costs scale $O(n^3)$ in the number of features, and the posterior geometry becomes complex, requiring longer chains.

Specifying the Model

PRIOR ON THE FUNCTION

$$f \sim \mathcal{GP}(0, k_\theta)$$

Zero-mean GP with RBF (squared exponential) kernel.

LIKELIHOOD

$$y_i \mid f(x_i), \sigma^2 \sim \mathcal{N}(f(x_i), \sigma^2)$$

Gaussian noise: $y = f(x) + \varepsilon$, with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$.

HYPERPRIORS

$$\alpha \sim LogNormal(0, 0.5)$$
$$\ell \sim LogNormal(0, 0.5)$$
$$\sigma \sim LogNormal(-1, 0.5)$$

Weakly informative priors on positive (α, ℓ, σ).

Hierarchical: $\alpha, \ell, \sigma \rightarrow f \rightarrow y$

Joint Distribution

Prior & Observations

$$\begin{bmatrix} y \\ f_* \end{bmatrix} \sim \mathcal{N}\left(0, \begin{bmatrix} K(X,X) + \sigma^2 I & K(X, X_*) \\ K(X_*, X) & K(X_*, X_*) \end{bmatrix}\right)$$

POSTERIOR PREDICTIVE AT A NEW INPUT · $f(x^*) \mid y \sim \mathcal{N}(\mu^*, v^*)$

POSTERIOR MEAN

$$\mu^* = k^{*T}(K_\theta + \sigma^2 I)^{-1} y$$

POSTERIOR VARIANCE

$$v^* = k(x^*, x^*) - k^{*T}(K_\theta + \sigma^2 I)^{-1} k^*$$

Stan Code

MODEL BLOCK

```
model {
  matrix[N_train, N_train] K;
  matrix[N_train, N_train] L_K;
  // Weakly informative priors for positive
  // GP hyperparameters
  alpha ~ lognormal(0, 0.5);
  rho   ~ lognormal(0, 0.5);
  sigma ~ lognormal(-1, 0.5);
  // Use robust built-in function to avoid
  // 0 * Inf gradient NaNs
  K = gp_exp_quad_cov(X_train, alpha, rho);
  for (i in 1:N_train) {
    K[i, i] = K[i, i] + square(sigma) + jitter;
  }
  L_K = cholesky_decompose(K);
  y_train ~ multi_normal_cholesky(
    rep_vector(0, N_train), L_K);
}
```

GENERATED QUANTITIES BLOCK

```
generated quantities {
  // Variable definitions redacted for brevity

  // Rebuild K using the built-in function
  K = gp_exp_quad_cov(X_train, alpha, rho);
  for (i in 1:N_train) {
    K[i, i] = K[i, i] + square(sigma) + jitter;
  }
  // Cross- and test covariance
  K_s  = gp_exp_quad_cov(X_train, X_test, alpha, rho);
  K_ss = gp_exp_quad_cov(X_test, alpha, rho);
  for (i in 1:N_test) {
    K_ss[i, i] = K_ss[i, i] + jitter;
  }
  // Posterior predictive mean and covariance
  alpha_vec   = mdivide_left_spd(K, y_train);
  f_test_mean = K_s' * alpha_vec;
  K_pred      = K_ss - K_s' * mdivide_left_spd(K, K_s);
  // Add observation noise + jitter to stabilize
  for (i in 1:N_test) {
    K_pred[i, i] = K_pred[i, i] + square(sigma) + jitter;
  }
  // Draw posterior predictive samples
  y_test_rep = multi_normal_rng(f_test_mean, K_pred);
}
```

Numeric Tricks for GP Stability

Ensuring Numerical Stability in GP Models

01. Safe Boundaries & Priors

Applied `<lower=1e-6>` constraints and Log-normal priors to prevent parameters from collapsing to zero.

02. Diagonal Jitter

Added `(1e-5)` to the covariance matrix to guarantee positive definiteness.

03. Optimized Built-in Functions

Utilized `gp_exp_quad_cov` to avoid 0 * Inf gradient crashes during HMC warmup exploration.

04. Cholesky & SPD Solvers

Replaced direct matrix inversions with robust Cholesky decompositions and `mdivide_left_spd` solvers.

Diagnostics

Summary Statistics (alpha, rho, sigma)

	Mean	StdDev	R_hat	ESS_bulk	ESS_tail
alpha	0.918281	0.233114	1.00542	1158.900	663.511
rho	0.389081	0.290146	1.00602	750.555	801.452
sigma	0.772572	0.059097	1.00153	1049.910	922.148

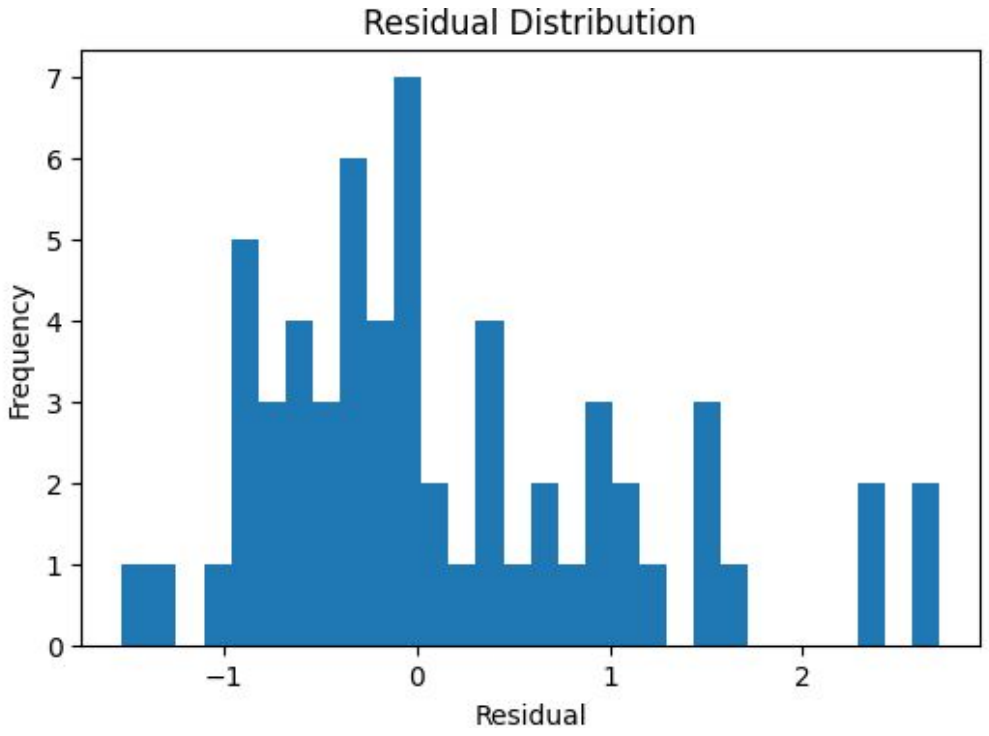
Quantile & Convergence Metrics

	Mean	StdDev	5%	50%	95%	R_hat
alpha	0.918281	0.233114	0.651976	0.873101	1.32777	1.00542
rho	0.389081	0.290146	0.096951	0.299800	1.01812	1.00602
sigma	0.772572	0.059097	0.669029	0.774810	0.86445	1.00153

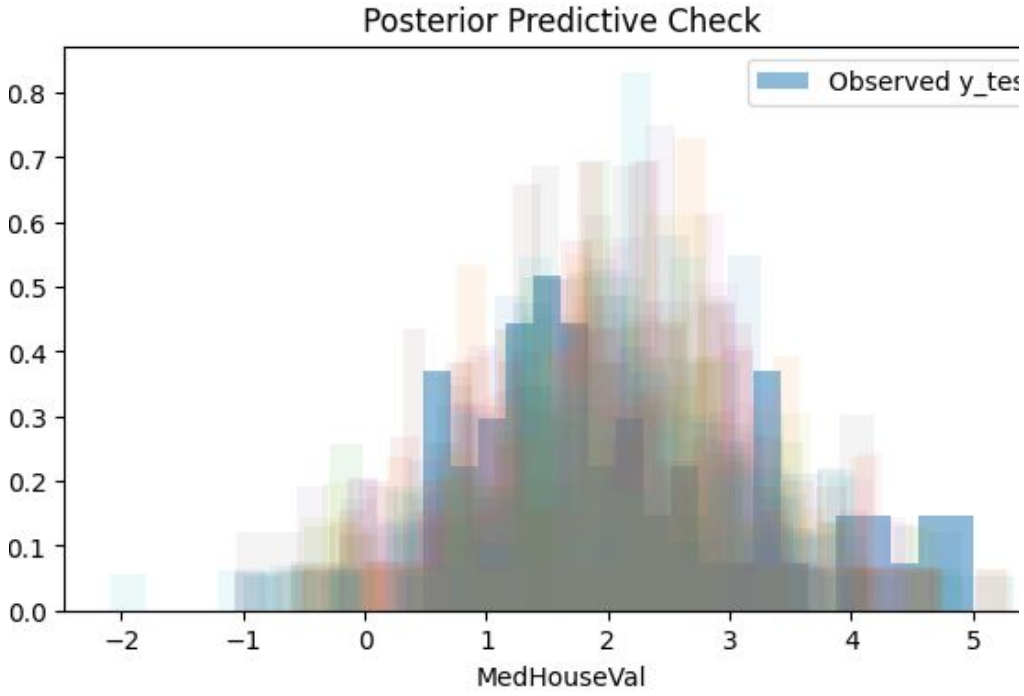
Gaussian Processes

Diagnostics

Residual Distribution

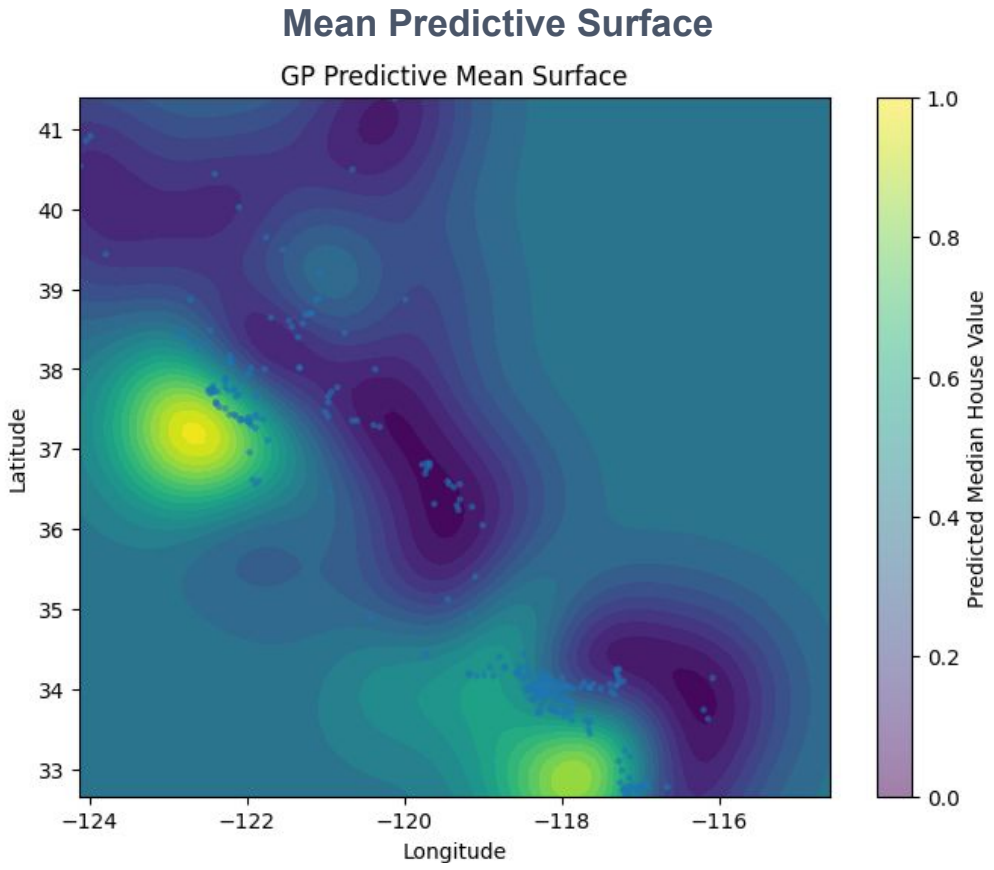


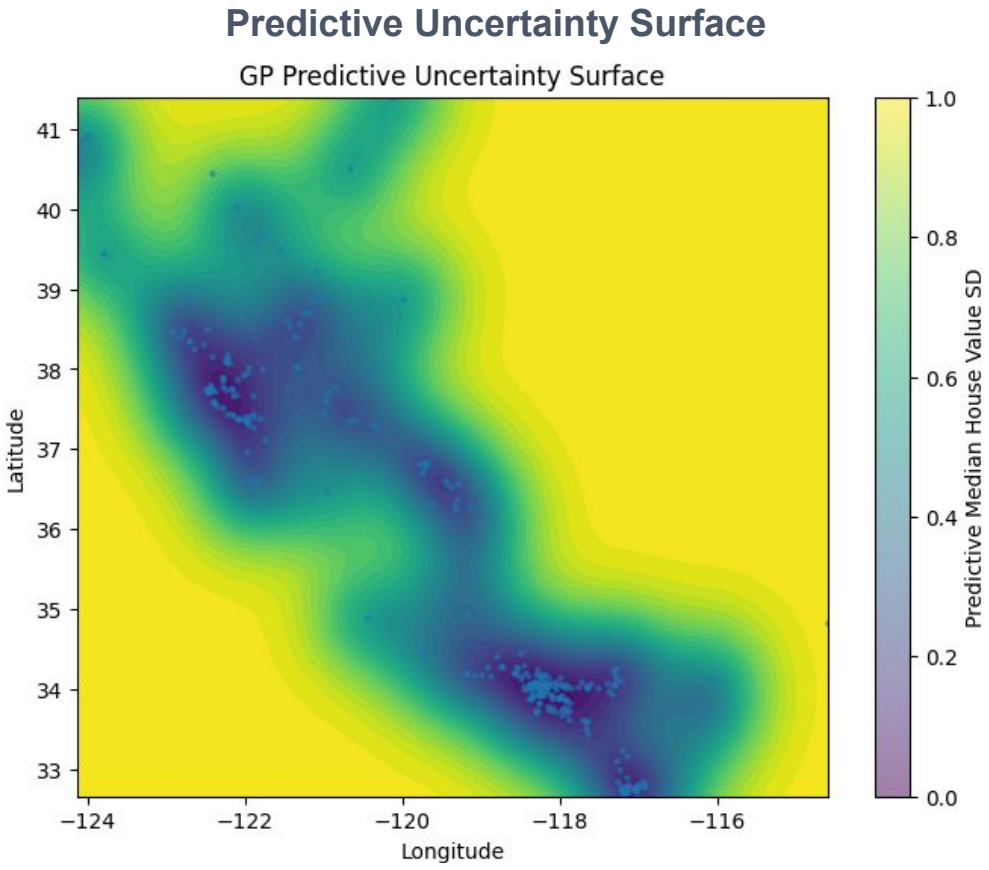
Posterior Predictive Check



Gaussian Processes

Reconstructed GP: Uncertainty Heatmap





Gaussian Processes

Areas for Improving the Model

Oversmoothing from the RBF kernel

The squared-exponential (RBF) kernel assumes a very smooth latent function modulated only by ρ, washing out sharp local changes and underestimating extreme prices.

$$k(x, x') = \alpha^2 \exp(-\|x - x'\|^2 / 2\rho^2)$$

Composite spatial + demographic kernels

Split x = (s, z) with s = (Lat, Lon) and combine two kernels — capturing smooth geography alongside nonlinear demographic dependence.

$$k(x, x') = k_{spatial}(s, s') + k_{demographic}(z, z')$$

A richer kernel with more features

Augment the input with all predictors (Lat, Lon, MedInc, HouseAge, AveRooms, AveBedrms, Pop, AveOccup) and use an Auto-Relevance Detector (ARD) kernel with dimension-specific length scales ρ□.

$$k(x, x') = \alpha^2 \exp(-\tfrac{1}{2} \Sigma_{\square} (x_{\square} - x'_{\square})^2 / \rho_{\square}^2)$$

Inclusion of linear structure

Combine a linear mean with a GP residual so large-scale linear effects (e.g., median income) are modeled directly while the GP captures nonlinear structure.

$$y_i = \beta_0 + x_i^T \beta + f(x_i) + \varepsilon_i$$

Thank You

Questions?